



CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

ASSIGNMENT - 02

Subject: Computer Networks

Subject Code: 24CST-339

Submitted by:

Name: Ajay Pratap Devra

UID: 24BAI70645

Branch/ Section: 24AIT_KRG-1_B

Submitted to:

Faculty Name: Alok Sir

Assignment – 2

Q1. What is an IP Address? Why do we need classes in case of IPV4 address? Explain each class, and mention how many host and networks are possible in each class. Also explain what is sub-netting and super-netting with examples.

Ans. An IP (Internet Protocol) address is a unique logical address assigned to a device connected to a network. It is used to identify a device and determine where data needs to be delivered.

For example: 192.168.1.10

An IPv4 address is 32 bits long and is divided into 4 octets, with each octet containing 8 bits.

192 . 168 . 1 . 10

8 bits 8 bits 8 bits 8 bits

Total = 32 bits

Each octet can have a value from 0 to 255.

IPv4 addresses were divided into different classes (A, B, C, D and E) to efficiently organize and allocate the available IP address space according to the size and requirements of different networks.

Different organizations need different numbers of IP addresses. For example, a large organization may require thousands of hosts, while a small organization may require only a few hundred. Classes provide different numbers of network bits and host bits, allowing networks of different sizes to be created.

Following are the different classes in IPv4 addresses:

CLASS A:

Range: 1.0.0.0 to 126.255.255.255

Default subnet mask: 255.0.0.0 or /8

Class A uses:

Network bits = 8

Host bits = 24

However, the first bit is fixed as 0, so only 7 bits are actually available for identifying different Class A networks.

So, number of networks = $2^7 = 128$

But 2 networks are reserved for i.e. 0 and 127, so we have 126 networks.

Number of hosts = $2^{24} - 2$ (since all host bits 0 is reserved for network address and all host bits 1 is reserved for broadcast address in all classes)

CLASS B:

Range: 128.0.0.0 to 191.255.255.255 Default

subnet mask: 255.255.0.0 or /16

Class B uses:

Network bits = 16

Host bits = 16

However, first 2 bits are fixed as 10 so only 14 bits are available for networks in class B.

So, number of networks = $2^{14} = 16,384$

Number of hosts = $2^{16} - 2$ CLASS

C:

Range: 192.0.0.0 to 223.255.255.255

Default subnet mask: 255.255.255.0 or /24

Class B uses:

Network bits = 24

Host bits = 8

However, first 2 bits are fixed as 110 so only 14 bits are available for networks in class B.

So, number of networks = $2^{14} = 16,384$

Number of hosts = $2^8 - 2$

CLASS D:

Range: 224.0.0.0 to 239.255.255.255

Class D is not divided into network and host portions like Classes A, B and C. It is used for multicast communication, where one sender can send data to multiple receivers simultaneously. For example, multicast can be useful for distributing the same data to a group of devices.

CLASS E:

Range: 240.0.0.0 to 255.255.255.255

Class E is reserved for experimental and research purposes. It is not used for normal host addressing.

Sub-netting is the process of dividing a large network into multiple smaller logical networks called subnets. It is done by borrowing bits from the host portion and using them as subnet/network bits.

For example

We have IP address: 192.168.1.0/24 Initially:

Network bits = 24

Host bits = 8

Suppose we borrow 2 host bits: /24 $\rightarrow$ /26 Now:

Network/Subnet bits = 26

Host bits = 6

Number of subnets: $2^{\text{bits borrowed}} = 2^2 = 4$

Hosts per subnet: $2^6 - 2 = 62$ The

four subnets are:

192.168.1.0/26

192.168.1.64/26

192.168.1.128/26

192.168.1.192/26

Thus, sub-netting allows one large network to be divided into smaller networks.

Advantages of sub-netting are: □ Better

utilization of IP addresses

- Reduces broadcast traffic
- Improves network management
- Provides logical separation between departments/networks

Super-netting is the process of combining multiple smaller networks into one larger logical network. It is the opposite of sub-netting. It is also called route aggregation or route summarization. It reduces the number of entries in routing tables.

Example: Suppose we have:

192.168.0.0/24

192.168.1.0/24

192.168.2.0/24

192.168.3.0/24

These four consecutive networks can be combined into:

192.168.0.0/22

The prefix changes from /24 to /22, meaning two bits have been removed from the network portion.

Q2. What is the difference between Link state and Distance vector routing? Explain how these two algorithms help in identifying the least cost path with the help of examples.

Ans. Distance Vector Routing is a routing method in which each router maintains a table containing the distance (cost) to different destinations and the next-hop router.

Each router shares its routing information with its neighbouring routers. Based on the information received, it updates its table and selects the path with the minimum total cost.

The Bellman-Ford algorithm is commonly associated with Distance Vector routing.

Example:

Consider the following network:

$$A - B = 2$$

$$B - D = 3$$

$$C - D = 1$$

$$A - C = 1$$

Suppose we want to find the least-cost path from A to D.

Bellman-Ford considers the distances through neighbouring routers.

Initially:

Distance from A:

$$A = 0$$

$$B = 2$$

$$C = 1$$

$$D = \infty$$

Now, using the neighbour information:

$$\text{Through B: } A \rightarrow B \rightarrow D = 2 + 3 = 5$$

Through C: $A \rightarrow C \rightarrow D = 1 + 1 = 2$

Therefore,

Shortest distance $A \rightarrow D = 2$

Path = $A \rightarrow C \rightarrow D$

Thus, Distance Vector finds the least-cost path by repeatedly relaxing routes using information received from neighbouring routers, as in the Bellman-Ford approach.

In Link State Routing, every router builds a complete map of the network topology by collecting information about all the links and their costs.

Each router:

- Identifies its neighbours and link costs.
- Shares this information with other routers.
- Builds a complete network topology.
- Uses the Dijkstra shortest-path algorithm to find the least-cost path.

Example:

$A - B = 2$

$B - D = 3$

$C - D = 1$

$A - C = 1$

To find the shortest path from A to D, Dijkstra's algorithm works by selecting the unvisited node with the smallest known distance.

Step 1: Start from A.

$A = 0$

$B = 2$

$C = 1$

$$D = \infty$$

The smallest unvisited distance is $C = 1$.

Step 2: Visit C and update D.

$C \rightarrow D = 1$ So:

$$A \rightarrow C \rightarrow D = 1 + 1 = 2$$

Now: $D = 2$

Step 3: D has the smallest remaining distance, so the algorithm reaches the destination.

Therefore:

Shortest distance $A \rightarrow D = 2$

Path = $A \rightarrow C \rightarrow D$

Feature	Distance Vector	Link State
Basic idea	Exchanges routing information with neighbours	Builds complete network topology
Algorithm	Bellman-Ford	Dijkstra
Network knowledge	Information from neighbours	Complete network map
Path calculation	Repeated distance updates	Selects nearest node and builds shortest-path tree
Convergence	Generally slower	Generally faster
Complexity	Simpler	More complex
Example	RIP	OSPF

Q3. What is congestion in a network? Why do we need to control congestion? Explain the difference between Leaky bucket and Token bucket with examples. Also, explain why do we need to do fragmentation?

Ans. Congestion occurs when the amount of data entering a network or router is greater than the network's capacity to process and forward it.

When too many packets arrive at the same time, they start waiting in buffers. If the buffer becomes full, packets may be dropped, causing delays and retransmissions.

Example:

Suppose a router can process 100 packets/second, but it receives 150 packets/second. The extra packets have to wait in the buffer. If this continues, the buffer may become full and packets will be dropped.

Congestion control is required to:

- Prevent packet loss and buffer overflow.
- Reduce network delay.
- Avoid unnecessary retransmissions.
- Maintain efficient network performance.
- Ensure that the traffic entering the network does not exceed its capacity.

Leaky Bucket Algorithm

The Leaky Bucket algorithm is a congestion-control/traffic-shaping technique that sends packets at a fixed and controlled rate, regardless of how quickly they arrive. It can be imagined as a bucket with a small hole at the bottom.

Example

Suppose:

Bucket capacity = 5 packets

Output rate = 2 packets/second

If 5 packets arrive suddenly, they are stored in the bucket and sent at a fixed rate of 2 packets/second.

If more packets arrive when the bucket is already full, the extra packets are discarded.

Thus, the Leaky Bucket smooths bursty traffic into a constant output flow.

Token Bucket Algorithm

In the Token Bucket algorithm, tokens are generated at a fixed rate and stored in a bucket. A packet can be transmitted only when it has the required number of tokens. Unlike the Leaky Bucket, it allows bursts of traffic when sufficient tokens have accumulated.

Example

Suppose:

Bucket capacity = 5 tokens

Tokens are generated at 2 tokens/second

If 5 tokens have accumulated, the sender can transmit a burst of packets using those tokens. After the tokens are consumed, the sender has to wait for new tokens to be generated.

Thus, Token Bucket controls the average transmission rate while allowing short bursts.

Feature	Leaky Bucket	Token Bucket
Output rate	Usually fixed	Can vary
Bursty traffic	Does not allow bursts	Allows bursts
Excess traffic	Discarded if bucket is full	Can wait for tokens
Main idea	Packets leak out at a fixed rate	Packets are sent when tokens are available
Traffic control	Smooths traffic	Controls average rate while allowing bursts

Fragmentation is the process of dividing a large IP packet into smaller packets (fragments) when the packet is larger than the Maximum Transmission Unit (MTU) of the network through which it needs to travel.

Different networks can have different MTU sizes.

Example: Suppose an IP packet is 4000 bytes, but the next network has an MTU of 1500 bytes.

The 4000-byte packet cannot be transmitted as a single packet through that network. Therefore, it has to be divided into smaller fragments.

The fragments are transmitted separately and, in IPv4, are reassembled at the destination.

We need fragmentation due to the following reasons: □

Different networks have different MTU sizes.

- A packet larger than the available MTU cannot cross that network as a single packet.
- Fragmentation allows the packet to be divided into smaller pieces that can fit the network's MTU.